Linux

Why Linux is Relevant Today

Linux has become incredibly important in today's technology landscape for several key reasons:

1. **Dominates critical infrastructure**: Linux runs 96% of the world's top servers, nearly all supercomputers, most cloud platforms (AWS, Google Cloud, Azure), and powers billions of Android devices.
2. **Open source advantage**: Being free and open source makes Linux cost-effective for businesses while allowing complete transparency and customization.
3. **Security and stability**: Linux's architecture provides better security through user permissions, regular updates, and community oversight, resulting in fewer vulnerabilities.
4. **Versatility**: Linux scales effectively from tiny IoT devices to massive enterprise servers, making it adaptable to virtually any computing need.
5. **Innovation platform**: Most cutting-edge technologies including AI, containers (Docker/Kubernetes), and cloud-native applications are built on Linux foundations.

Future Importance

Linux and CLI skills will continue growing in value because:

1. **Cloud computing growth**: As cloud adoption increases, Linux skills become more essential since most cloud infrastructure runs on Linux.
2. **DevOps revolution**: Modern development practices rely heavily on CLI tools for deployment, testing, and infrastructure management.
3. **Career opportunities**: Professionals with strong Linux command line skills command higher salaries and have excellent job prospects.
4. **AI and data science**: These rapidly growing fields rely extensively on Linux environments and command line tools.
5. **Fundamental skill**: Understanding CLI develops critical thinking about how computer systems actually work, creating more adaptable IT professionals.

The command line isn't just surviving in a GUI world—it's thriving as the essential interface for modern computing infrastructure.

Navigation Commands

pwd

Purpose: Print Working Directory - shows the current directory path

Example:

```
$ pwd
/home/user/documents
```

Explanation: This command shows you are currently in the `/home/user/documents` directory. It's useful when you need to confirm your current location in the filesystem.

cd

Purpose: Change Directory - navigate between folders

Examples:

```
# Go to home directory
$ cd
# or
$ cd ~

# Go to specific directory
$ cd /var/log

# Go up one directory
$ cd ..

# Go to previous directory
$ cd -

# Go to subdirectory
$ cd documents

# Go to absolute path
$ cd /home/user/downloads
```

Explanation: The `cd` command allows you to move around the filesystem. Without arguments, it takes you to your home directory. With `..` it moves up one level, with a directory name it moves into that directory.

ls

Purpose: List directory contents

Examples:

```
# Basic listing
$ ls
Documents  Downloads  Pictures  Music

# Long listing format
$ ls -l
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Documents
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Downloads
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Pictures
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Music

# Show hidden files
$ ls -a
.  ..  .bash_history  Documents  Downloads  Pictures  Music  .config

# Combine options
$ ls -la
total 40
drwxr-xr-x 5 user user 4096 Apr 10 14:22 .
drwxr-xr-x 3 root root 4096 Apr 10 13:15 ..
-rw------- 1 user user  562 Apr 10 14:21 .bash_history
drwxr-xr-x 3 user user 4096 Apr 10 14:20 .config
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Documents
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Downloads
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Pictures
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Music

# List with human-readable file sizes
$ ls -lh
drwxr-xr-x 2 user user 4.0K Apr 10 14:22 Documents
drwxr-xr-x 2 user user 4.0K Apr 10 14:22 Downloads

# Long listing with time-sorted (newest first) and all files
```

```
$ ls -ltra
total 40
drwxr-xr-x 3 root root 4096 Apr 10 13:15 ..
-rw------- 1 user user  562 Apr 10 14:21 .bash_history
drwxr-xr-x 3 user user 4096 Apr 10 14:20 .config
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Documents
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Downloads
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Pictures
drwxr-xr-x 2 user user 4096 Apr 10 14:22 Music
drwxr-xr-x 5 user user 4096 Apr 10 14:22 .
```

Explanation:

- `ls` by itself shows visible files/directories
- `-l` shows long format with permissions, owner, size, date
- `-a` shows hidden files (starting with .)
- `-h` shows sizes in human-readable format (KB, MB, etc.)
- `-t` sorts by modification time (newest first)
- `-r` reverses sort order
- These options can be combined: `-ltra` combines long format, time sorting, reverse order, and showing all files

File Operations

touch

Purpose: Create empty files or update timestamps of existing files

Examples:

```
# Create a new empty file
$ touch newfile.txt

# Create multiple files at once
$ touch file1.txt file2.txt file3.txt

# Update timestamp of existing file
$ touch existingfile.txt
```

Explanation: `touch` creates files if they don't exist or updates the access and modification times if they do exist. It's a simple way to create empty files.

mkdir

Purpose: Create directories

Examples:

```
# Create a directory
$ mkdir documents

# Create nested directories using -p
$ mkdir -p projects/website/css

# Create multiple directories
$ mkdir dir1 dir2 dir3
```

Explanation: `mkdir` creates directories. The `-p` option allows creating parent directories as needed (if they don't exist).

File Creation Methods

There are several ways to create files in Linux:

1. Using touch:

```
$ touch filename.txt
```

2. Using echo with redirection:

```
$ echo "This is content" > file.txt
```

3. Using cat with redirection:

```
$ cat > newfile.txt
This is line 1
```

```
This is line 2
(Press Ctrl+D to save and exit)
```

4. Using an editor:

```
$ nano filename.txt
$ vim filename.txt
```

5. Using heredoc:

```
$ cat << EOF > script.sh
#!/bin/bash
echo "Hello World"
EOF
```

Explanation: These methods offer different ways to create files - from empty files (touch) to files with specific content (echo, cat, editors).

cp

Purpose: Copy files and directories

Examples:

```
# Copy a file
$ cp source.txt destination.txt

# Copy a file to a directory
$ cp myfile.txt /home/user/documents/

# Copy recursively (for directories)
$ cp -r source_directory/ destination_directory/

# Preserve file attributes when copying
$ cp -p original.txt copy.txt

# Verbose mode to see what's being copied
$ cp -v file.txt backup/file.txt
'file.txt' -> 'backup/file.txt'
```

Explanation: The `cp` command copies files or directories. The `-r` option is needed to copy directories recursively. `-p` preserves file permissions and timestamps.

mv

Purpose: Move or rename files and directories

Examples:

```
# Rename a file
$ mv oldname.txt newname.txt

# Move a file to a different directory
$ mv file.txt /home/user/documents/

# Move and rename in one step
$ mv old/path/file.txt new/path/renamed.txt

# Move multiple files
$ mv file1.txt file2.txt directory/

# Move with verbose output
$ mv -v oldfile.txt newfile.txt
'oldfile.txt' -> 'newfile.txt'
```

Explanation: `mv` can both move files to new locations and rename them. Unlike `cp`, it doesn't need a special option to move directories.

rm

Purpose: Remove files and directories

Examples:

```
# Remove a file
$ rm file.txt

# Remove multiple files
$ rm file1.txt file2.txt
```

```
# Remove with confirmation prompt
$ rm -i important.txt
rm: remove regular file 'important.txt'? y

# Remove recursively (for directories)
$ rm -r directory/

# Force removal without prompts
$ rm -f unwanted.txt

# Combine force and recursive (use with caution!)
$ rm -rf old_directory/

# Verbose output
$ rm -v temp.txt
removed 'temp.txt'
```

Explanation: `rm` removes files and directories. Be careful with this command, especially with the `-rf` option, as it removes directories and their contents without asking for confirmation.

File Content Operations

cat

Purpose: Concatenate and display file contents

Examples:

```
# Display file contents
$ cat file.txt
This is the content of file.txt

# Combine multiple files
$ cat file1.txt file2.txt
Content of file1
Content of file2

# Create a file with content
$ cat > newfile.txt
Type content here
Press Ctrl+D when done
```

```
# Append to a file
$ cat >> existing.txt
This will be added to the end
Press Ctrl+D when done

# Show line numbers
$ cat -n script.sh
     1  #!/bin/bash
     2  echo "Hello World"
```

Explanation: `cat` displays the contents of files or combines multiple files. It can also be used with redirection to create or append to files.

echo

Purpose: Display text or variables, often used with redirection

Examples:

```
# Display text
$ echo "Hello World"
Hello World

# Create a file with content
$ echo "This is new content" > file.txt

# Append to a file
$ echo "Additional line" >> file.txt

# Display environment variables
$ echo $HOME
/home/user

# Echo with no trailing newline
$ echo -n "No newline"

# Interpret backslash escapes
$ echo -e "Line 1\nLine 2"
Line 1
Line 2
```

Explanation:

- `echo` displays text to the terminal
- `>` redirects output, overwriting any existing file
- `>>` appends output to a file
- `-n` suppresses the trailing newline
- `-e` enables interpretation of backslash escapes like `\n` (newline)

less

Purpose: View file contents page by page

Examples:

```
# View a file
$ less largefile.txt

# View with line numbers
$ less -N largefile.txt
```

Navigation in less:

- `Space` or `f`: page down
- `b`: page up
- `/pattern`: search forward for pattern
- `n`: next occurrence of pattern
- `N`: previous occurrence of pattern
- `g`: go to start of file
- `G`: go to end of file
- `q`: quit

Explanation: `less` is useful for viewing large files as it doesn't load the entire file into memory. It provides scrolling, searching, and other navigation features.

head

Purpose: Display the beginning of files

Examples:

```
# Show first 10 lines (default)
$ head file.txt

# Show first n lines
$ head -n 5 file.txt
Line 1
Line 2
Line 3
Line 4
Line 5

# Show multiple files
$ head -n 3 file1.txt file2.txt
==> file1.txt <==
Line 1 of file1
Line 2 of file1
Line 3 of file1

==> file2.txt <==
Line 1 of file2
Line 2 of file2
Line 3 of file2

# Show first n bytes
$ head -c 20 file.txt
First 20 characters
```

Explanation: `head` shows the beginning of a file. By default, it shows the first 10 lines, but you can specify a different number with `-n`.

tail

Purpose: Display the end of files

Examples:

```
# Show last 10 lines (default)
$ tail file.txt

# Show last n lines
$ tail -n 5 file.txt
```

```
Line 96
Line 97
Line 98
Line 99
Line 100

# Follow file updates in real-time
$ tail -f /var/log/syslog

# Show last n bytes
$ tail -c 20 file.txt

# Show all but the first n lines
$ tail -n +5 file.txt  # Start from line 5 to end
```

Explanation: `tail` shows the end of a file. The `-f` option is particularly useful for monitoring log files as they're being updated.

Text Processing

grep

Purpose: Search for patterns in files

Examples:

```
# Basic search
$ grep "pattern" file.txt

# Case insensitive search
$ grep -i "pattern" file.txt

# Recursive search in directories
$ grep -r "pattern" /path/to/directory/

# Show line numbers
$ grep -n "pattern" file.txt
5:This line contains the pattern we're looking for

# Show only filenames with matches
$ grep -l "pattern" *.txt
```

```
# Invert match (lines NOT matching)
$ grep -v "pattern" file.txt

# Count matches
$ grep -c "pattern" file.txt
7

# Match whole words only
$ grep -w "word" file.txt

# Extended regular expressions
$ grep -E "pattern1|pattern2" file.txt
```

Explanation: `grep` searches for text patterns in files. It's extremely versatile with many options for refining searches.

awk

Purpose: Text processing tool for tabular data

Examples:

```
# Print specific columns (fields)
$ awk '{print $1, $3}' file.txt

# Process with custom field separator
$ awk -F: '{print $1, $3}' /etc/passwd

# Filter rows based on condition
$ awk '$3 > 50 {print $1, $3}' data.txt

# Sum values in a column
$ awk '{sum += $2} END {print sum}' numbers.txt

# Count lines
$ awk 'END {print NR}' file.txt

# Custom processing with if statements
$ awk '{if ($3 > 100) print $1 " is expensive"}' prices.txt
```

Explanation: `awk` is powerful for processing structured text, especially tabular data. It treats each line as a record with fields separated by whitespace (by default).

sed

Purpose: Stream editor for transforming text

Examples:

```
# Replace first occurrence on each line
$ sed 's/old/new/' file.txt

# Replace all occurrences on each line
$ sed 's/old/new/g' file.txt

# Replace on specific line numbers
$ sed '3s/old/new/' file.txt  # Only on line 3

# Delete specific lines
$ sed '5d' file.txt  # Delete line 5
$ sed '1,5d' file.txt  # Delete lines 1-5

# Insert text after a pattern
$ sed '/pattern/a\New line after pattern' file.txt

# Insert text before a pattern
$ sed '/pattern/i\New line before pattern' file.txt

# Save changes to file (in-place edit)
$ sed -i 's/old/new/g' file.txt

# Multiple operations
$ sed -e 's/old/new/g' -e 's/foo/bar/g' file.txt
```

Explanation: `sed` is used for text transformations like search and replace, insertion, and deletion. The `-i` option modifies files in-place.

find

Purpose: Search for files and directories

Examples:

```
# Find files by name
$ find /path/to/search -name "filename.txt"

# Case insensitive name search
$ find /path/to/search -iname "*.jpg"

# Find directories only
$ find /path/to/search -type d

# Find files only
$ find /path/to/search -type f

# Find files modified in last n days
$ find /path/to/search -mtime -7  # Last 7 days

# Find files by size
$ find /path/to/search -size +10M  # Larger than 10MB

# Find and execute a command on each result
$ find /path/to/search -name "*.tmp" -exec rm {} \;

# Find files with specific permissions
$ find /path/to/search -perm 644

# Find files owned by specific user
$ find /path/to/search -user username
```

Explanation: `find` is powerful for locating files based on various criteria like name, type, size, and modification time. The `-exec` option allows actions on found files.

Editors

nano

Purpose: Simple, beginner-friendly text editor

Examples:

```
# Open or create a file
```

```
$ nano file.txt

# Open with cursor at specific line
$ nano +10 file.txt
```

Common nano shortcuts:

- `Ctrl+G`: Display help
- `Ctrl+O`: Save file
- `Ctrl+X`: Exit
- `Ctrl+K`: Cut line
- `Ctrl+U`: Paste line
- `Ctrl+W`: Search for text
- `Ctrl+\`: Search and replace

Explanation: Nano is an easy-to-use editor ideal for beginners. It displays shortcuts at the bottom of the screen.

vim

Purpose: Powerful, modal text editor

Examples:

```
# Open or create a file
$ vim file.txt

# Open at specific line
$ vim +10 file.txt

# Open in read-only mode
$ vim -R file.txt
```

Basic vim modes:

- Normal mode (Esc): for navigating and commands
- Insert mode (i): for inserting text
- Command mode (:): for operations like save and quit

Common vim commands:

- `:q`: Quit
- `:w`: Save
- `:wq` or `:x`: Save and quit
- `:q!`: Quit without saving
- `i`: Enter insert mode
- `Esc`: Return to normal mode
- `/pattern`: Search forward
- `n`: Next occurrence
- `dd`: Delete line
- `yy`: Copy (yank) line
- `p`: Paste

Explanation: Vim has a steeper learning curve but is extremely powerful once mastered. It operates in different modes for different operations.

System Monitoring

top

Purpose: Dynamic real-time system monitor

Example:

```
$ top

Output example:
top - 14:35:23 up 5 days, 2:03, 2 users, load average: 0.07, 0.02, 0.00
Tasks: 213 total,   1 running, 212 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.0 us,  0.5 sy,  0.0 ni, 98.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem:   7950.4 total,   7097.8 free,    435.1 used,    417.5 buff/cache
MiB Swap:  2048.0 total,   2048.0 free,      0.0 used.   7232.7 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 1234 user      20   0  743972  42380  29876 S   1.0   0.5   0:02.03 firefox
 5678 root      20   0   43816   3936   3168 S   0.3   0.0   1:23.45 systemd
```

Interactive commands in top:

- `q`: Quit
- `k`: Kill a process (asks for PID)

- `r`: Renice a process (change priority)
- `f`: Fields to display
- `o`: Change sort order
- `1`: Toggle individual CPU display

Explanation: `top` shows a dynamic view of system processes and resource usage. It updates regularly and allows interactive commands to filter or act on processes.

ps

Purpose: Report process status

Examples:

```
# Show current user's processes
$ ps
  PID TTY          TIME CMD
 2345 pts/0    00:00:01 bash
 3456 pts/0    00:00:00 ps

# Show all processes with full details
$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0 168004 12224 ?        Ss   Apr10   0:01 /sbin/init
root           2  0.0  0.0      0     0 ?        S    Apr10   0:00 [kthreadd]
user        1243  0.1  0.5 743972 42380 ?        Sl   14:20   0:02 firefox

# Show process tree
$ ps -ejH
  PID  PGID   SID TTY          TIME CMD
    1     1     1 ?        00:00:01 systemd
   23    23    23 ?        00:00:00 systemd-journald
  435   435   435 ?        00:00:00 sshd
  789   789   789 ?        00:00:00 sshd
  812   812   812 pts/0    00:00:00 bash

# Show process by PID
$ ps -p 1234
  PID TTY          TIME CMD
 1234 ?        00:00:02 firefox
```

Explanation: `ps` shows snapshots of current processes. Unlike `top`, it's static rather than continuously updating. Common option combinations:

- `ps aux`: Show all processes
- `ps -ef`: Full listing, common on Unix systems
- `ps -ejH`: Show process hierarchy (parent-child relationships)

kill

Purpose: Send signals to processes

Examples:

```
# Terminate a process (SIGTERM)
$ kill 1234

# Force kill (SIGKILL)
$ kill -9 1234
# or
$ kill -KILL 1234

# List available signals
$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1

# Send HUP signal (often used to reload configs)
$ kill -HUP 1234

# Kill by process name using pkill
$ pkill firefox

# Kill all processes for a user
$ pkill -u username
```

Explanation: `kill` sends signals to processes. The default signal is SIGTERM (15), which asks the process to terminate gracefully. SIGKILL (9) forcefully terminates a process but doesn't allow it to clean up.

df

Purpose: Report filesystem disk space usage

Examples:

```
# Show all filesystems
$ df
Filesystem     1K-blocks     Used Available Use% Mounted on
/dev/sda1       41251136 10123456  29127680  26% /
tmpfs            4096000      800   4095200   1% /dev/shm

# Human-readable sizes
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        40G  9.7G   28G  26% /
tmpfs           4.0G  800K  3.9G   1% /dev/shm

# Show filesystem type
$ df -T
Filesystem     Type     1K-blocks     Used Available Use% Mounted on
/dev/sda1      ext4      41251136 10123456  29127680  26% /
tmpfs          tmpfs      4096000      800   4095200   1% /dev/shm

# Show specific filesystem
$ df -h /home
```

Explanation: `df` shows disk space usage for mounted filesystems. The `-h` option displays sizes in human-readable format (GB, MB, etc.).

du

Purpose: Estimate file and directory space usage

Examples:

```
# Show size of current directory
$ du -s
45876    .

# Human-readable sizes
$ du -sh
45M    .

# Show subdirectory sizes
$ du -h --max-depth=1
```

```
20M     ./documents
15M     ./downloads
10M     ./pictures
45M     .

# Show specific directory or file
$ du -sh /home/user/documents
20M     /home/user/documents

# Sort by size (largest last)
$ du -h --max-depth=1 | sort -h
10M     ./pictures
15M     ./downloads
20M     ./documents
45M     .
```

Explanation: `du` estimates file space usage. Unlike `df`, which shows filesystem usage, `du` calculates the size of directories and their contents.

systemctl

Purpose: Control the systemd system and service manager

Examples:

```
# Check status of a service
$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor
preset: enabled)
   Active: active (running) since Sat 2023-04-10 14:30:12 UTC; 2h 5min ago

# Start a service
$ sudo systemctl start nginx

# Stop a service
$ sudo systemctl stop nginx

# Restart a service
$ sudo systemctl restart nginx
```

```
# Enable service to start at boot
$ sudo systemctl enable nginx

# Disable service from starting at boot
$ sudo systemctl disable nginx

# List all active services
$ systemctl list-units --type=service

# Check if a service is enabled
$ systemctl is-enabled nginx
enabled

# Reload systemd after changes to unit files
$ sudo systemctl daemon-reload
```

Explanation: `systemctl` is used to control and manage systemd, the system and service manager for Linux. It's used to start, stop, and check the status of system services.

User Management

useradd

Purpose: Create a new user

Examples:

```
# Basic user creation
$ sudo useradd akash

# Create user with specific home directory
$ sudo useradd -m -d /home/akash akash

# Create user with specific shell
$ sudo useradd -m -s /bin/bash akash

# Create user with comment (full name)
$ sudo useradd -m -c "Akash Sharma" akash

# Create user with specific UID
$ sudo useradd -u 1500 akash
```

```
# Create user in specific group
$ sudo useradd -g developers akash

# Create user with multiple secondary groups
$ sudo useradd -m -G developers,admins,users akash

# Create user with account expiry date
$ sudo useradd -e 2023-12-31 akash

# Create system user (for daemons)
$ sudo useradd -r nginx

# Create user with specific login shell but no login ability
$ sudo useradd -s /sbin/nologin webservices
```

Explanation: `useradd` creates new user accounts. The `-m` flag creates the home directory, which isn't created by default in many distributions.

userdel

Purpose: Delete a user account

Examples:

```
# Delete user but keep home directory
$ sudo userdel akash

# Delete user and home directory
$ sudo userdel -r akash

# Force deletion even if user is still logged in
$ sudo userdel -f akash
```

Explanation: `userdel` removes user accounts. The `-r` option also removes the user's home directory and mail spool.

groupadd

Purpose: Create a new group

Examples:

```
# Create a new group
$ sudo groupadd developers

# Create group with specific GID
$ sudo groupadd -g 1500 developers

# Create system group
$ sudo groupadd -r nginx
```

Explanation: `groupadd` creates new groups, which can be used to organize users and control access to files.

groupdel

Purpose: Delete a group

Example:

```
$ sudo groupdel developers
```

Explanation: `groupdel` removes a group from the system. It will fail if the group is a primary group for any user.

passwd

Purpose: Change user password

Examples:

```
akash PS 2023-04-10 0 99999 7 -1 (Password set, SHA512 # Change your own password)
$ passwd
Current password:
New password:
Retype new password:

# Change another user's password (as root)
$ sudo passwd akash
New password:
```

```
Retype new password:

# Lock a user account
$ sudo passwd -l akash

# Unlock a user account
$ sudo passwd -u akash

# Delete a user's password (make it empty)
$ sudo passwd -d akash

# Get password status
$ sudo passwd -S akash
```

Explanation: `passwd` changes passwords for user accounts. Root can change any user's password without knowing the current one.

chage

Purpose: Change user password expiry information

Examples:

```
# View current password aging information
$ sudo chage -l akash
Last password change                                    : Apr 10, 2023
Password expires                                        : Jul 09, 2023
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 7

# Set account expiration date
$ sudo chage -E 2023-12-31 akash

# Force password change on next login
$ sudo chage -d 0 akash

# Set minimum days between password changes
```

```
$ sudo chage -m 7 akash

# Set maximum password age (days)
$ sudo chage -M 90 akash

# Set warning period before password expiration
$ sudo chage -W 7 akash
```

Explanation: chage modifies password aging policies for users. It's useful for enforcing password security policies.

Root Permissions and User Management

Purpose: The root user (superuser) has complete control over the system

Key characteristics:

- User ID (UID) of 0
- Unlimited system privileges
- Can access any file and run any command
- Can modify system configurations
- Can create, modify, and delete users
- Not restricted by file permissions

Importance of Root:

- Required for system administration tasks
- Needed for installing system-wide software
- Manages services and daemons
- Configures hardware and network settings
- Can repair system issues
- Maintains system security

Dangers of Root:

- No safety net - commands execute without restriction
- Easy to accidentally damage the system
- Security risk if compromised
- No logging/auditing built in
- Best practice: only use root access when necessary

Sudo (Superuser Do)

Purpose: Execute commands with root privileges temporarily

Examples:

```
# Install a package
$ sudo apt install nginx

# Edit a system file
$ sudo nano /etc/hosts

# View logs that require elevated permissions
$ sudo tail /var/log/auth.log

# Run multiple commands with sudo privileges
$ sudo -i
[sudo] password for user:
root@hostname:~# apt update
root@hostname:~# apt upgrade
root@hostname:~# exit

# Execute a command without password prompt (if configured)
$ sudo -n systemctl restart nginx
```

Explanation:

- sudo provides temporary root privileges for specific commands
- Users must be in the sudo/wheel group or listed in /etc/sudoers
- Advantages over direct root login:
    - Logs which user executed privileged commands
    - Limits scope of elevated permissions
    - Requires password re-entry after timeout (usually 15 minutes)
    - Can be configured to allow only specific commands

User Switching Commands

Examples:

```
# Switch to root user (need root password)
$ su
Password:
root@hostname:/home/user#

# Switch to root user with root's environment
$ su -
Password:
root@hostname:~#

# Switch to root using sudo (need your password)
$ sudo su
[sudo] password for user:
root@hostname:/home/user#

# Switch to root with root's environment using sudo
$ sudo su -
[sudo] password for user:
root@hostname:~#

# Switch to another user
$ su john
Password:
john@hostname:/home/user$

# Switch to another user with their environment
$ su - john
Password:
john@hostname:~$

# Switch to another user with sudo (no password needed for target user)
$ sudo su - john
[sudo] password for user:
john@hostname:~$

# Execute a command as another user
$ su - john -c "ls -la"
```

```
# or
$ sudo -u john ls -la
```

Explanation of differences:

- `su` alone: Switches to root but keeps your current environment variables, working directory
- `su -` (or `su -l`): Switches to root with a login shell (loads root's environment, changes to root's home directory)
- `sudo su`: Becomes root using your password (instead of root's password)
- `sudo -i`: Similar to `sudo su -`, provides a root login shell

Configuring Sudo Access

Purpose: Control which users can use sudo and what commands they can run

Main configuration file:

```
# Edit sudoers file safely
$ sudo visudo

Common sudoers configurations:
# Allow a user to run all commands
username ALL=(ALL:ALL) ALL

# Allow a user to run specific commands without password
username ALL=(ALL:ALL) NOPASSWD: /sbin/shutdown, /sbin/reboot

# Allow members of a group to run all commands
%admin ALL=(ALL:ALL) ALL
%sudo ALL=(ALL:ALL) ALL

# Allow user to run commands as a specific user
username ALL=(postgres) /usr/bin/psql

# Include additional configuration files
#includedir /etc/sudoers.d
```

Explanation:

- Always use `visudo` to edit sudo configuration (it checks for syntax errors)
- Format: `USER HOST=(RUN_AS) COMMANDS`
- `ALL` is a wildcard meaning "all users", "all hosts", or "all commands"
- `NOPASSWD:` allows executing commands without password prompt
- ***Group names are prefixed with `%`***

Best Practices for Root Access

1. **Limit use of root account**
    - Use sudo for individual commands instead of staying logged in as root
    - Set a strong root password
2. **Configure sudo properly**
    - Grant only necessary permissions to users
    - Use command restrictions where possible
    - Log sudo commands for accountability
3. **User switching safety**
    - After becoming root, be extremely careful with commands
    - Exit privileged sessions when no longer needed
    - Avoid using root for routine tasks
4. **Secure shared environments**
    - Be aware of other users with sudo access
    - Audit sudo logs regularly: `sudo grep sudo /var/log/auth.log`
    - Use `sudo -l` to check what privileges you have
5. **Understand the risks**
    - Root has no restrictions and can completely destroy the system
    - A typo in a command can have serious consequences
    - Always double-check destructive commands (rm, dd, etc.)

File Permissions

chmod

Purpose: Change file permissions

Examples:

```
# Using octal notation
$ chmod 755 script.sh  # rwxr-xr-x

# Using symbolic notation
$ chmod u+x script.sh  # Add execute permission for user
$ chmod g+w file.txt   # Add write permission for group
$ chmod o-rwx file.txt # Remove all permissions for others
$ chmod a+r file.txt   # Add read permission for all (user, group, others)

# Recursive permission change
$ chmod -R 755 directory/  # Change permissions for directory and all
contents

# Reference file permissions
$ chmod --reference=reference.txt target.txt  # Copy permissions
```

Explanation:

- chmod changes file permissions
- Octal notation uses numbers: 4 (read), 2 (write), 1 (execute)
- Common combinations: 755 (rwxr-xr-x), 644 (rw-r--r--), 700 (rwx------)
- Symbolic notation uses: u (user/owner), g (group), o (others), a (all)
- Operations: + (add), - (remove), = (set exactly)
- Permissions: r (read), w (write), x (execute)

chown

Purpose: Change file owner and group

Examples:

```
# Change owner
$ sudo chown user1 file.txt

# Change owner and group
$ sudo chown user1:group1 file.txt

# Change only the group
$ sudo chown :group1 file.txt
```

```
# Recursive ownership change
$ sudo chown -R user1:group1 directory/

# Reference file ownership
$ sudo chown --reference=reference.txt target.txt
```

Explanation: chown changes the owner and/or group of files and directories. Only the root user or the current owner can change a file's ownership.

Bash Shell and Commands

Bash Shell Basics

Purpose: Command processor that typically runs in a text window where you can type commands

Basic Shell Features:

```
# Command history
$ history
  1  ls -la
  2  cd /etc
  3  cat passwd

# Navigate command history
# Press Up/Down arrow keys to cycle through previous commands

# Command editing
# Use left/right arrows, Home, End, etc. to navigate within a command

# Tab completion
$ cd Doc[TAB]  # Completes to "cd Documents/"

# Exit the shell
$ exit
```

Explanation: Bash (Bourne Again SHell) is the default shell on most Linux distributions. It provides command history, tab completion, and command editing features.

Bash Environment Variables

Purpose: Store system and user-defined values for shell usage

Examples:

```
# Display environment variables
$ env

# Display specific variable
$ echo $HOME
/home/user

# Set a variable (session only)
$ MYVAR="Hello World"
$ echo $MYVAR
Hello World

# Export a variable to child processes
$ export MYVAR="Hello World"

# Add to PATH variable
$ export PATH=$PATH:/new/directory
```

Explanation: Environment variables store system information and can be used in scripts and commands. Common ones include `HOME`, `PATH`, `USER`, and `SHELL`.

Bash Scripting

Purpose: Automate tasks by writing scripts

Simple script example:

```
#!/bin/bash
# My first script

echo "Hello, world!"
echo "Current user: $USER"
echo "Current directory: $PWD"

# Variables
NAME="Linux User"
```

```
echo "Hello, $NAME"

# Input
echo "What's your name?"
read USERNAME
echo "Nice to meet you, $USERNAME"

# Conditionals
if [ -f /etc/passwd ]; then
    echo "The passwd file exists"
else
    echo "The passwd file does not exist"
fi

# Loops
for i in 1 2 3 4 5; do
    echo "Number: $i"
done
```

Running a script:

```
# Make script executable
$ chmod +x script.sh

# Run script
$ ./script.sh
```

Explanation: Bash scripts automate repetitive tasks. They start with `#!/bin/bash` (shebang line), can use variables, conditionals, loops, and more.

Common Bash Shortcuts

Purpose: Increase efficiency when working in the shell

Examples:

```
# Clear screen
Ctrl + L
```

```
# End of line
Ctrl + E

# Beginning of line
Ctrl + A

# Cut text from cursor to end of line
Ctrl + K

# Paste previously cut text
Ctrl + Y

# Search command history
Ctrl + R

# Stop current command
Ctrl + C

# Suspend current command (resume with fg)
Ctrl + Z

# Logout
Ctrl + D
```

Explanation: These keyboard shortcuts make it faster to navigate and edit commands in the bash shell.

Bash Command Chaining

Purpose: Run multiple commands sequentially or conditionally

Examples:

```
# Sequential execution (run commands in order)
$ command1 ; command2 ; command3

# Conditional execution (command2 runs only if command1 succeeds)
$ command1 && command2

# Conditional execution (command2 runs only if command1 fails)
$ command1 || command2
```

```
# Command substitution (use output of a command)
$ echo "Today is $(date)"

# Pipes (send output of command1 as input to command2)
$ command1 | command2

# Background execution
$ long-running-command &
```

Explanation: Command chaining allows for complex operations by combining simple commands. Pipes are especially powerful for creating command pipelines.

Bash Redirections

Purpose: Control command input and output

Examples:

```
# Redirect stdout to a file (overwrite)
$ command > output.txt

# Redirect stdout to a file (append)
$ command >> output.txt

# Redirect stderr to a file
$ command 2> error.txt

# Redirect both stdout and stderr to a file
$ command > output.txt 2>&1

# Redirect stdin from a file
$ command < input.txt

# Discard output
$ command > /dev/null
```

Explanation: Redirections control where command input comes from and where output goes. This is useful for logging, capturing output, or providing input from files.

Bash Aliases

Purpose: Create shortcuts for commonly used commands

Examples:

```
# Create an alias
$ alias ll='ls -la'

# Use the alias
$ ll

# List all aliases
$ alias

# Remove an alias
$ unalias ll

# Permanent aliases (add to ~/.bashrc)
$ echo 'alias ll="ls -la"' >> ~/.bashrc
```

Explanation: Aliases save time by creating shortcuts for frequently used commands. Adding them to `~/.bashrc` makes them permanent across sessions.

Archiving

tar

Purpose: Archive files and directories

Examples:

```
# Create a tar archive
$ tar -cf archive.tar file1 file2 directory/

# Create a compressed tar archive (with gzip)
$ tar -czf archive.tar.gz file1 file2 directory/

# Create a compressed tar archive (with bzip2)
$ tar -cjf archive.tar.bz2 file1 file2 directory/

# Create a compressed tar archive (with xz)
$ tar -cJf archive.tar.xz file1 file2 directory/
```

```
# Extract a tar archive
$ tar -xf archive.tar

# Extract a compressed tar archive (automatic compression detection)
$ tar -xf archive.tar.gz

# List contents of an archive
$ tar -tf archive.tar

# Extract specific files from an archive
$ tar -xf archive.tar file1 directory/file2

# Extract to a different directory
$ tar -xf archive.tar -C /path/to/extract/

# Add files to an existing archive
$ tar -rf archive.tar newfile

# Create archive with verbose output
$ tar -cvf archive.tar directory/
```

Explanation:

- `tar` is used to create, maintain, and extract archives
- Common options:
  - `-c`: Create a new archive
  - `-x`: Extract files from an archive
  - `-f`: Specify archive filename
  - `-t`: List contents of an archive
  - `-v`: Verbose output
  - `-z`: Compress with gzip
  - `-j`: Compress with bzip2
  - `-J`: Compress with xz
  - `-r`: Append files to an archive
  - `-C`: Change to specified directory before performing operations

System Files

/etc/passwd

Purpose: Stores basic user account information

Example content:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
user1:x:1000:1000:akashsharma,,,:/home/user1:/bin/bash
```

Field explanation (colon-separated):

1. Username
2. Password placeholder (actual encrypted password is in /etc/shadow)
3. User ID (UID)
4. Group ID (GID)
5. User description/comment field
6. Home directory
7. Login shell

Viewing safely:

```
$ cat /etc/passwd
$ grep user1 /etc/passwd
```

Explanation: `/etc/passwd` contains basic user account information and is readable by all users. For security, passwords are stored in `/etc/shadow`.

/etc/shadow

Purpose: Stores encrypted password information

Example content:

```
root:$6$xyz123...long_hash:18000:0:99999:7:::
user1:$6$abc456...long_hash:18120:0:99999:7:::
```

Field explanation (colon-separated):

1. Username
2. Encrypted password hash
3. Days since Jan 1, 1970 that password was last changed
4. Days before password may be changed (0 means any time)
5. Days after which password must be changed
6. Days before password expiration to warn user
7. Days after password expires that account is disabled
8. Days since Jan 1, 1970 that account is disabled
9. Reserved field

Viewing safely (requires root):

```
$ sudo cat /etc/shadow
$ sudo grep user1 /etc/shadow
```

Explanation: `/etc/shadow` contains encrypted passwords and is only readable by root. It improves security by separating password data from the publicly readable `/etc/passwd` file.

Inodes

What are inodes

Purpose: Store metadata about files and directories

Information stored in inodes:

- File type (regular file, directory, symbolic link, etc.)
- Permissions
- Owner and group IDs
- File size
- Timestamps (access, modification, change)
- Number of hard links
- Pointers to data blocks
- But NOT the filename or actual content

Explanation: An inode is a data structure in Linux/Unix filesystems that stores all information about a file except its name and actual data. Each file and directory has a unique inode number within a filesystem. Multiple filenames (hard links) can point to the same inode.

Checking inodes

Examples:

```
# Show inode information
$ ls -i file.txt
1234567 file.txt

# Show detailed inode information
$ stat file.txt
  File: file.txt
  Size: 1024       Blocks: 8          IO Block: 4096   regular file
Device: 8,1        Inode: 1234567     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   user)   Gid: ( 1000/   user)
Access: 2023-04-10 12:34:56.789012345 +0000
Modify: 2023-04-10 12:34:56.789012345 +0000
Change: 2023-04-10 12:34:56.789012345 +0000
 Birth: -

# Find files with the same inode number (hard links)
$ find /path/to/search -inum 1234567

# Check inode usage
$ df -i
Filesystem      Inodes  IUsed   IFree IUse% Mounted on
/dev/sda1      2621440 218122 2403318    9% /
```

Explanation:

- Each file uses one inode
- Filesystems have a limited number of inodes
- Running out of inodes prevents creating new files even if disk space is available
- Hard links share the same inode number
- Symbolic links use their own inode but point to another file

SSH

ssh-keygen

Purpose: Generate authentication key pairs for SSH

Examples:

```
# Generate default key pair
$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/user/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/user/.ssh/id_rsa
Your public key has been saved in /home/user/.ssh/id_rsa.pub

# Generate key with specific algorithm and bits
$ ssh-keygen -t ed25519

# Generate key with custom filename
$ ssh-keygen -f ~/.ssh/custom_key

# Generate key with comment
$ ssh-keygen -C "user@example.com"

# Generate key with no passphrase in non-interactive mode
$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/automated_key -N ""

# Show fingerprint of key
$ ssh-keygen -l -f ~/.ssh/id_rsa
```

Explanation:

- `ssh-keygen` creates a key pair: private key (e.g., `id_rsa`) and public key (`id_rsa.pub`)
- The private key stays on your computer and should never be shared
- The public key can be placed on remote servers to enable passwordless login
- A passphrase adds extra security to your private key
- Common key types: RSA, Ed25519, ECDSA

ssh

Purpose: Connect securely to remote systems

Examples:

```
# Basic connection
$ ssh username@hostname
$ ssh username@192.168.1.100
```

```
# Specify port
$ ssh -p 2222 username@hostname

# Use specific identity file (private key)
$ ssh -i ~/.ssh/custom_key username@hostname

# Run a command on remote server without login shell
$ ssh username@hostname "ls -la /var/log"

# Port forwarding (local to remote)
$ ssh -L 8080:localhost:80 username@hostname

# Port forwarding (remote to local)
$ ssh -R 8080:localhost:80 username@hostname

# Jump host (proxy through an intermediate server)
$ ssh -J jumpuser@jumphost username@destination

# Enable X11 forwarding
$ ssh -X username@hostname

# Verbose output for debugging
$ ssh -v username@hostname
```

Copy public key to remote server:

```
$ ssh-copy-id username@hostname
```

SSH configuration file (~/.ssh/config):

```
Host myserver
    HostName 192.168.1.100
    User admin
    Port 2222
    IdentityFile ~/.ssh/custom_key

# Then connect with just:
$ ssh myserver
```

Explanation:

- SSH (Secure Shell) provides encrypted communication between computers
- Key-based authentication is more secure than passwords
- `ssh-copy-id` copies your public key to a server's authorized_keys file
- The SSH config file simplifies connection to frequently used servers
- Port forwarding allows secure access to services through SSH tunnels